

### Essay 3: Feature Engineering Impact

Feature engineering refers to the process of transforming raw data into meaningful inputs that make patterns easier for a model to learn. Even when using sophisticated algorithms, poor features can limit model performance, while well-designed features can significantly improve results—even with simpler models. In many practical machine learning settings, especially those involving structured tabular data, feature engineering determines how much usable signal the model receives. My work with the Nigeria hospital dataset demonstrated this clearly.

At the start of my project, the available features were extremely minimal: patient age, arrival and departure timestamps, satisfaction scores, and the service the patient received. Most of these variables, in their raw form, contained very little predictive structure for the task of forecasting the service a patient would be assigned at admission. For example, the logistic regression model trained on only the raw numerical and date fields produced accuracy barely above random guessing for a four-class problem. This highlighted that the model was not struggling because of lack of algorithmic sophistication but because the features were simply not informative enough.

Feature engineering addresses this issue by creating new representations of the existing variables so that the model can detect relevant patterns. For example, I converted the raw age values into clinically meaningful buckets such as child, young adult, adult, middle-aged, and elderly. Grouping ages helped the model pick up differences that a single numerical age value tends to conceal. Similarly, a continuous variable like age doesn't act the same across all patients; different age groups follow different patterns. This extra structure gave the model a clearer way of understanding patient characteristics.

Temporal variables provided even richer opportunities. The raw `arrival_date` column was almost useless on its own, but once decomposed into components such as month, quarter, day of week, weekend flags, part of month, and season (dry vs rainy), the model could finally learn patterns relating to service usage across time. For example, weekend arrivals or rainy-season visits may correlate with increases in emergency service utilisation. These engineered features allowed the model to understand seasonality and hospital workflow patterns without needing a specialised time-aware model.

Domain knowledge played a crucial role in deciding what features to include and which to exclude. Understanding what information is available at admission—and what would introduce leakage—was essential. Variables such as `departure_date` or satisfaction scores occur after the service assignment and therefore cannot be used as model inputs. Including them would give an inflated sense of performance because the model would be relying on information that it would not realistically see at prediction time. Avoiding these leakage-prone fields required an understanding of both the clinical process and the data-generating workflow.

The final engineered feature set was built strictly from information available at admission, and this included a set of temporal features. After one-hot encoding, the model had a richer and more structured input space. Performance improved modestly but consistently: logistic regression increased from around 0.22 accuracy with raw features to about 0.24 with the engineered set, and the `RandomForestClassifier` reached around 0.26 accuracy. Although the absolute gains were small—reflecting that the dataset itself contained limited signal for this prediction task—the improvements came from better features, not from switching algorithms.

This reinforces an important lesson: when the underlying predictive signal is weak, no algorithm can compensate for insufficient or uninformative features. Conversely, when the features are thoughtfully designed and reflect the real-world processes behind the data, even simple models can perform competitively. Automated feature engineering tools can be helpful, but in constrained, domain-specific settings like hospital admissions, manual feature design informed by domain understanding remains

essential.

In summary, this project confirmed that feature engineering often has more influence on model performance than choosing a more complex algorithm. Models learn only from the inputs they are given, and transforming raw fields into structured, meaningful features provides the most significant performance gains. Even though the dataset had natural limitations, the clearest improvements came from careful feature engineering rather than algorithm choice.